

Coreset Technique

(Part 1)

OUTLINE

- ① Coresets and Composable Coresets
- ② Case study: k-center clustering
 - Basic notions on clustering
 - k-center problem
 - k-center as a coreset primitive
- ③ Other applications (Part 2)

Coresets and Composable Coresets

Coreset technique

Suppose that we want to solve a problem Π on instances P which are too large to be processed by known algorithms.

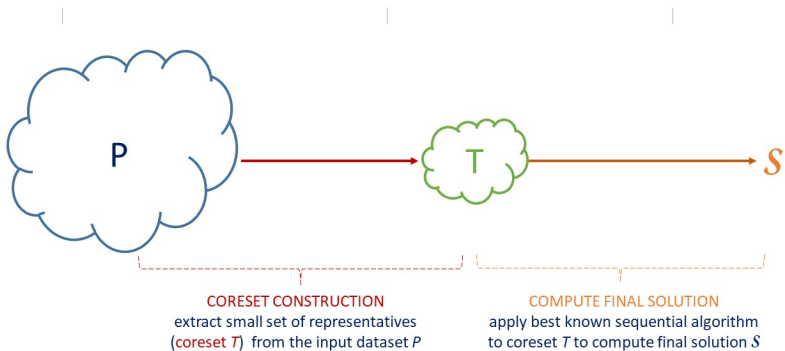
Coreset technique

- 1 Extract a "small" subset T from P (dubbed **coreset**), making sure that it represents P well, w.r.t. solutions to Π .
- 2 Run best known (possibly slow) sequential algorithm for Π on the small coreset T , rather than on the entire input P .

The technique is effective if

- T can be extracted efficiently by processing P , possibly in a distributed or streaming fashion.
- The solution computed on T is a good solution for Π w.r.t. the entire input P .

Coreset technique



Composable Coreset technique

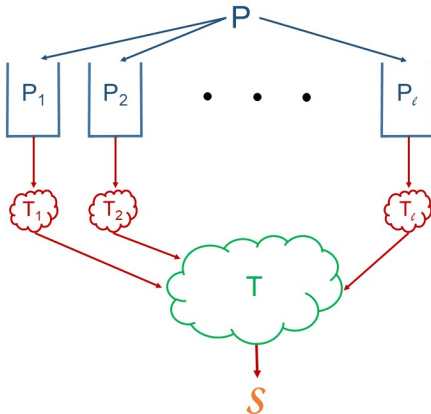
Composable Coreset technique

- 1 Partition P into ℓ subsets $P_1, P_2, \dots, P_\ell$, and extract a "small" coreset T_i from each P_i , making sure that it represents P_i well.
- 2 Run best known (possibly slow) sequential algorithm for Π on $T = \cup_{i=1,\ell} T_i$, rather than on P .

The technique is effective if

- Each T_i can be extracted efficiently from P_i in parallel for all i 's.
- The final coreset T is still small and the solution computed on T is a good solution for Π w.r.t. the entire input P .

Composable coreset technique



- Extract a *small coreset* T_i from each P_i
- Apply best known sequential algorithm to final coreset $T =$ union of the T_i 's, to compute final solution S

Case study: k-center clustering

General definition of clustering

CLUSTERING groups a set of objects into several subsets (**clusters**) such that

- objects in the same cluster are "**similar**" to one another
- objects in different clusters are "**dissimilar**" from one another

The definition of a **clustering problem** requires

- A notion of **similarity**, usually captured by a **distance function** on the object space
- An **objective to optimize**, selecting the best among several solutions, **and/or constraints to satisfy**

Example

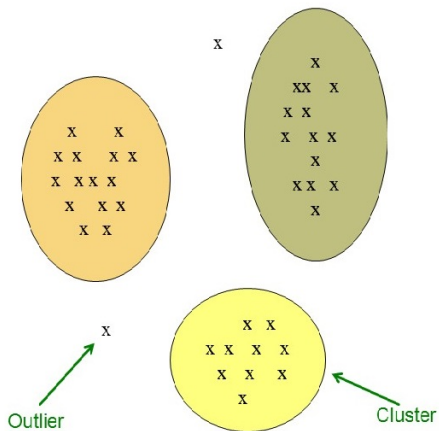


Figure from [Leskovec et al.: Mining Massive Datasets, 2014].

The clustering problem is a hard one!

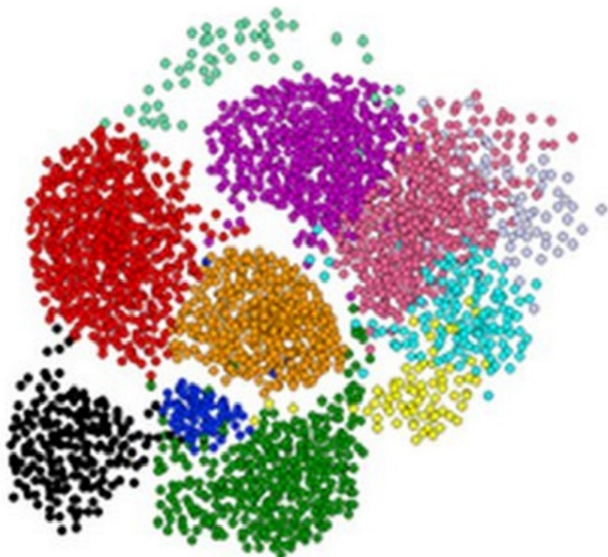


Figure from [Leskovec et al.: Mining Massive Datasets, 2014].

Importance of clustering

Main uses of clustering:

- ① Assessing the structure of the data
- ② Assigning classes to data for future prediction tasks
- ③ Summarizing and compressing data

Applications in many different fields. Some examples:

- **Marketing:** to segment (potential) customers.
- **Biology:** to identify protein families.
- **Image processing:** to do object recognition.
- **Information retrieval:** to categorize documents or web pages.
- **Network analysis:** to identify communities.
- **Facility location:** to decide where to open network hubs/facilities.

Why is clustering a relevant case study for us?

Metric Space

Typically, the input of a clustering problem consists of a set of points from a **metric space**

Definition

A **metric space** is an ordered pair (M, d) where M is a set and $d(\cdot)$ is a **metric** on M , i.e., a function

$$d : M \times M \rightarrow \mathbb{R}$$

such that for every $x, y, z \in M$ the following holds

- $d(x, y) \geq 0$;
- $d(x, y) = 0$ if and only if $x = y$;
- $d(x, y) = d(y, x)$; (symmetry)
- $d(x, z) \leq d(x, y) + d(y, z)$; (triangle inequality)

Remark: *The choice of the function may have a significant impact on the effectiveness of the cluster analysis.*

Triangle inequality is useful for big data analysis

Distance functions

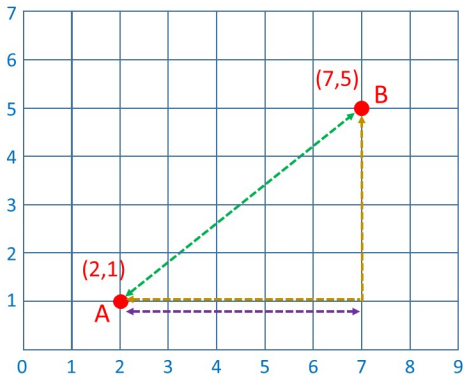
Minkowski distances. Let $X, Y \in \mathbb{R}^n$, with $X = (x_1, x_2, \dots, x_n)$ and $Y = (y_1, y_2, \dots, y_n)$. For $r > 0$ the L_r - distance between X and Y (a.k.a. L_r - norm)

$$d_{L_r}(X, Y) = \left(\sum_{i=1}^n |x_i - y_i|^r \right)^{1/r}.$$

- $r = 2$ (*Euclidean distance*): standard distance in $\mathbb{R}^n$ (also denoted by $\|\cdot\|$)
- $r = 1$ (*Manhattan distance*): used in grid-like environments
- $r = \infty$ (*Chebyshev distance*): maximum absolute differences of coordinates (i.e., the limit for r that tends to ∞).

Distance functions

Examples of Minkowski distances:



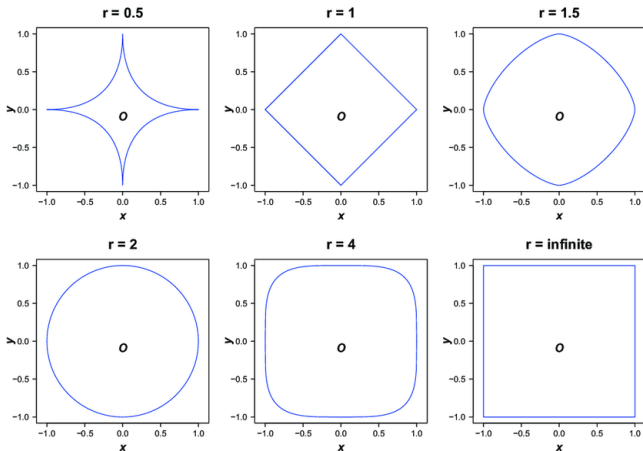
$$d_{l_1}(A,B) = (7-2) + (5-1) = 9$$

$$d_{l_2}(A,B) = (5^2 + 4^2)^{1/2} = 6.403..$$

$$d_{l_\infty}(A,B) = \max\{5,4\} = 5$$

Distance functions

Comparison of Minkowski distances:



Points at distance ≤ 1 from center

Picture from: Urquiza-Aguilar et al., Empirical Analysis of the Minkowski Distance Order in Geographical Routing Protocols for VANETs. Proc. WWIC, 2015.

Distance functions

Angular distance. It is used when points are vectors in $\mathbb{R}^n$. Given $X = (x_1, x_2, \dots, x_n)$ and $Y = (y_1, y_2, \dots, y_n)$ in $\mathbb{R}^n$, their **angular distance** is the angle between them, that is,

$$\begin{aligned}d_{\text{angular}}(X, Y) &= \arccos\left(\frac{X \cdot Y}{\|X\| \cdot \|Y\|}\right) \\&= \arccos\left(\frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n (x_i)^2} \sqrt{\sum_{i=1}^n (y_i)^2}}\right) \in [0, \pi]\end{aligned}$$

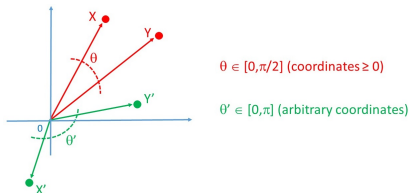
Example: For $X = (1, 2, -1)$ and $Y = (2, 1, 1)$

$$d_{\text{angular}}(X, Y) =$$

Obs: In [LRU14] it is called *cosine distance*, but in the literature the term has a different meaning.

Distance functions

For non-negative coordinates, $d_{\text{angular}}(X, Y) \in [0, \pi/2]$, while for arbitrary coordinates the range is $[0, \pi]$.



To normalize values in $[0, 1]$, $d_{\text{angular}}(X, Y)$ is often divided by π or $\pi/2$ depending whether vectors have arbitrary or non-negative coordinates. Also, to satisfy the second property of metric spaces, **scalar multiples of a vector are regarded as the same vector**.

Remark. *This distance is used in information retrieval for documents.* A document X over a vocabulary of n words can be represented as an n -vector, where x_i is the number of occurrences of the i -th word of the vocabulary in X .

Distance functions

- **Hamming distance.** Used when points are *binary vectors* over some n -dimensional space: $X, Y \in \{0, 1\}^n$. The Hamming distance between two vectors is **the number of coordinates in which they differ**.

$$d_{\text{Hamming}}(X, Y) = |\{i | x_i \neq y_i\}|$$

Example: for $X = (0, 1, 1, 0, 1)$ and $Y = (1, 1, 1, 0, 0)$

$$d_{\text{Hamming}}(X, Y) =$$

Observation: $d_{\text{Hamming}}(X, Y) = d_{L1}(X, Y)$

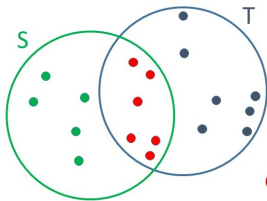
Distance functions

- **Jaccard distance**. Used when points are *sets* (e.g., documents seen as bags of words). Let S and T be two sets over the same ground set of elements. The **Jaccard distance** between S and T is

$$d_{\text{Jaccard}}(S, T) = 1 - \frac{|S \cap T|}{|S \cup T|} = \frac{|S \cup T| - |S \cap T|}{|S \cup T|}.$$

Note that the distance ranges in $[0, 1]$, and it is 0 iff $S = T$ and 1 iff S and T are disjoint. The value $|S \cap T|/|S \cup T|$ is referred to as the *Jaccard similarity* of the two sets.

Example:



$$d_{\text{Jaccard}}(S, T) = 1 - 6/18 = 2/3$$

Which distance should we use?

Minkowski and Angular distances are used when an object is characterized by the numerical values of its features (e.g., frequency).

- **Minkowski distance**: it aggregates the gap in each dimension between two objects. Examples: GPS coordinates; measurements of physical/chemical processes.
- **Angular distance**: it measures the ratios among features' values, rather than their absolute values. Example: distance between documents of different lengths, where each document is characterized by the relative frequency of words from a given vocabulary.

Which distance should we use?

Hamming and Jaccard distances are used when an object is characterized by having or not having some features (no multiplicity).

- **Hamming distance**: it measures the total number of differences between two objects. Examples: fixed-length binary words, genetic distance.
- **Jaccard distance**: it measures the ratio between the number of differences between two sets and the total number of points in the sets. Example: the topics describing a document.

Observation: All of the above distances satisfy the four requirements for a metric space.

Exercise

Show that the L_1 satisfies the four requirements for a metric space.

Types of clusterings

Given a set of points in a metric space, a clustering problem often specifies an **objective function** to optimize.

The objective function also allows to compare different solutions.

Clustering problems can be categorized based on whether or not

- A target number k of clusters is given in input.
- For each cluster a **center** must be identified.
- Disjoint clusters are sought.

Background: optimization problem

Combinatorial optimization problem

A **(COMBINATORIAL) OPTIMIZATION PROBLEM** is defined by

- A set $\mathcal{I}$ of **instances** (i.e., possible inputs)
- A set $\mathcal{S}$ of **solutions** (i.e., possible outputs)
- For each $i \in \mathcal{I}$, a set $\mathcal{S}_i \subseteq \mathcal{S}$ of **feasible solutions**
- **objective function** $\Phi : \mathcal{S} \rightarrow \mathbb{R}$

Given $i \in \mathcal{I}$ the problem requires to find an **OPTIMAL SOLUTION**, i.e., a feasible solution $s^* \in \mathcal{S}_i$ such that

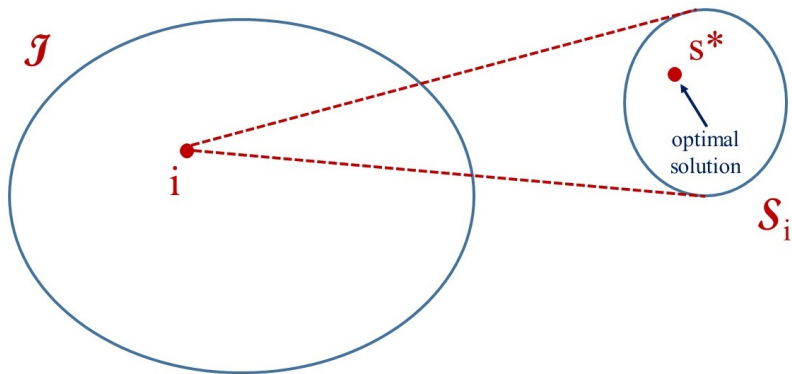
- $\Phi(s^*) = \min\{\Phi(s) : s \in \mathcal{S}_i\}$ (*minimization problem*)

or

- $\Phi(s^*) = \max\{\Phi(s) : s \in \mathcal{S}_i\}$ (*maximization problem*)

Obs. Technically, if $\mathcal{S}_i$ is not finite, **min** and **max** become **inf** and **sup**.

Background: optimization problem



Background: approximation algorithm

Approximation algorithm

Let Π be an optimization problem. For $c \geq 1$, a **c -approximation algorithm** A for Π is an algorithm that for each instance $i \in \mathcal{I}$ returns a feasible solution $A(i) \in \mathcal{S}_i$ such that

- $\Phi(A(i)) \leq c \cdot \min\{\Phi(s) : s \in \mathcal{S}_i\}$ (minimization problem)

or

- $\Phi(A(i)) \geq (1/c) \cdot \max\{\Phi(s) : s \in \mathcal{S}_i\}$ (maximization problem)

We refer to c as **approximation ratio**, and to $A(i)$ as **c -approximate solution** for i , or **c -approximation** of the optimal solution for i .

Observations

Approximation algorithms play a **crucial role** in big data computing for the following reasons

- For several important optimization problems **finding optimal solutions for large inputs is very time consuming**. E.g.,
 - NP-hard problems for which it is likely that **exponential time** is required.
 - Problems for which polynomial-time algorithms exist but require **superlinear number of operations** (e.g., quadratic, cubic).
- **Allowing for approximations widens the spectrum of possible solutions** and makes it easier to find one.
- In real-world instances, often affected by noise or uncertainty, **the true optimum may not be well defined**.

Center-based clustering

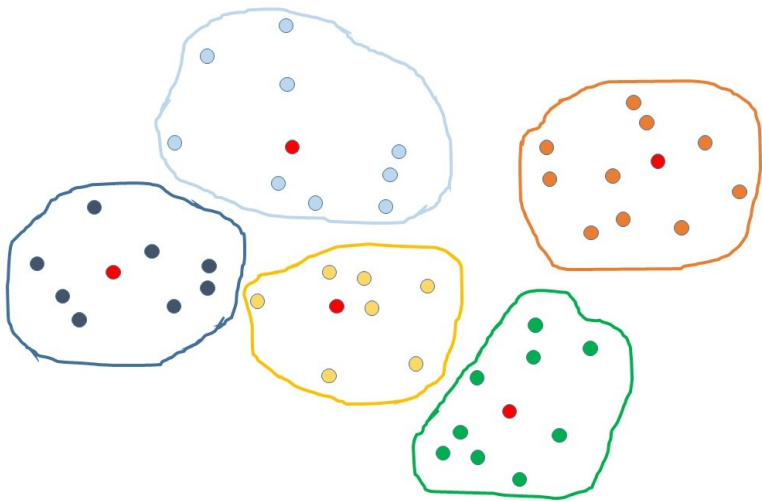
Let P be a set of N points in metric space (M, d) , and let k be the target number of clusters, $1 \leq k \leq N$.

A k -clustering of P is a tuple $\mathcal{C} = (C_1, C_2, \dots, C_k; S)$ which comprises

- A partition of $P = C_1 \cup C_2 \cup \dots \cup C_k$ into k clusters, with $C_i \cap C_j = \emptyset, \forall i, j$
- A set $S = \{c_1, c_2, \dots, c_k\}$ of k centers, where $c_i \in C_i$ is the center of cluster C_i , for $1 \leq i \leq k$.

Remark: in the above definition, centers are points of P . In some cases (e.g., for Euclidean spaces), centers outside P may be allowed.

Example of 5-clustering



Popular center-based clustering problems

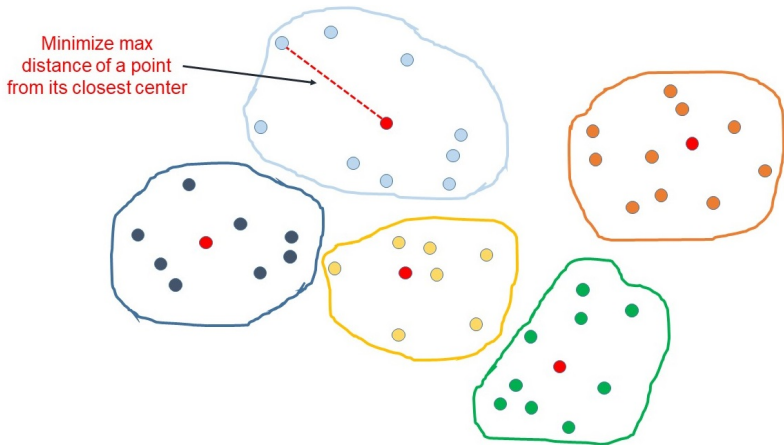
There are 3 popular problems that require to determine center-based clusterings for a given input P , which optimize one of 3 objectives.

Specifically:

- **k-center clustering:** aims at *minimizing the maximum distance of any point from the center of its cluster*
- **k-means clustering:** aims at *minimizing the sum of the squared distances of the points from the centers of their respective clusters* (a.k.a. Sum of Squared Errors (SSE))
- **k-median clustering:** aims at *minimizing the sum of the distances of the points from the centers of their respective clusters*

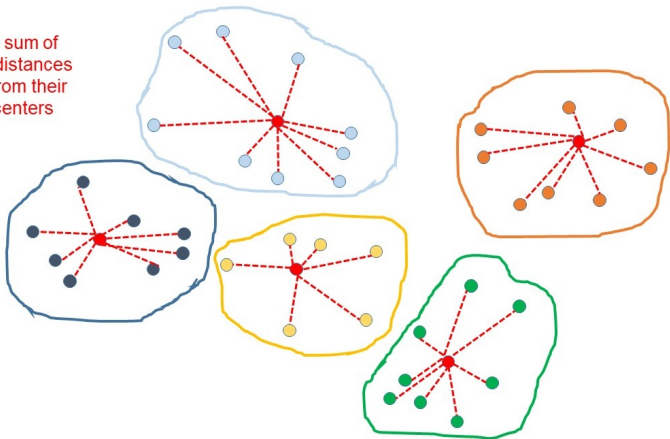
Observation: for k-means and k-median, minimizing the sum of (squared) distances is equivalent to minimizing the average (squared) distance.

k-center objective



k-means/median objectives

Minimize sum of
(squared) distances
of points from their
closest centers



Formal definitions of the problems

For any point $x \in P$ and any subset $S \subseteq P$ we define

$$d(x, S) = \min_{y \in S} d(x, y)$$

i.e., the distance between x and the closest point of S

Definition

Let (M, d) be a metric space. The **k-center/k-means/k-median clustering problems** are optimization problems that, given a finite pointset $P \subseteq M$ and an integer $k \leq |P|$, require to return the **subset $S \subseteq P$ of k centers** which minimizes the following respective objective functions:

- $\Phi_{\text{kcenter}}(P, S) = \max_{x \in P} d(x, S)$ (k-center clustering)
- $\Phi_{\text{kmeans}}(P, S) = \sum_{x \in P} (d(x, S))^2$ (k-means clustering).
- $\Phi_{\text{kmedian}}(P, S) = \sum_{x \in P} (d(x, S))$ (k-median clustering).

Terminology and notations

For the k-center/means/median clustering problems we have that:

INSTANCE (P, k)

FEASIBLE SOLUTIONS: All subsets $S \subseteq P$ of size k

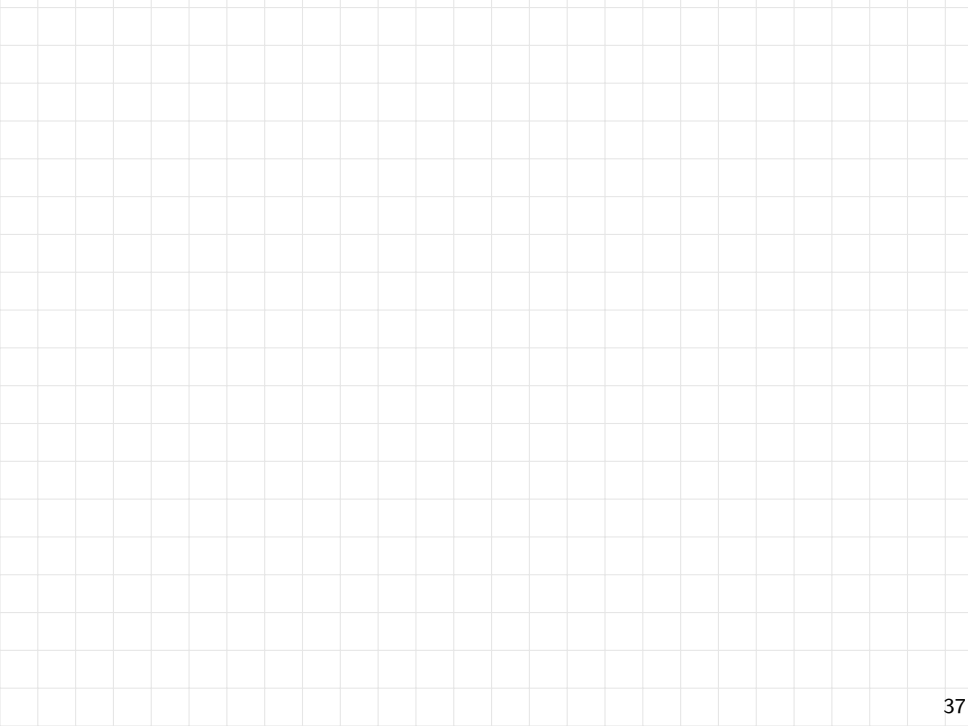
OPTIMAL SOLUTION: S^* (i.e., the feasible solution which minimizes the appropriate objective function)

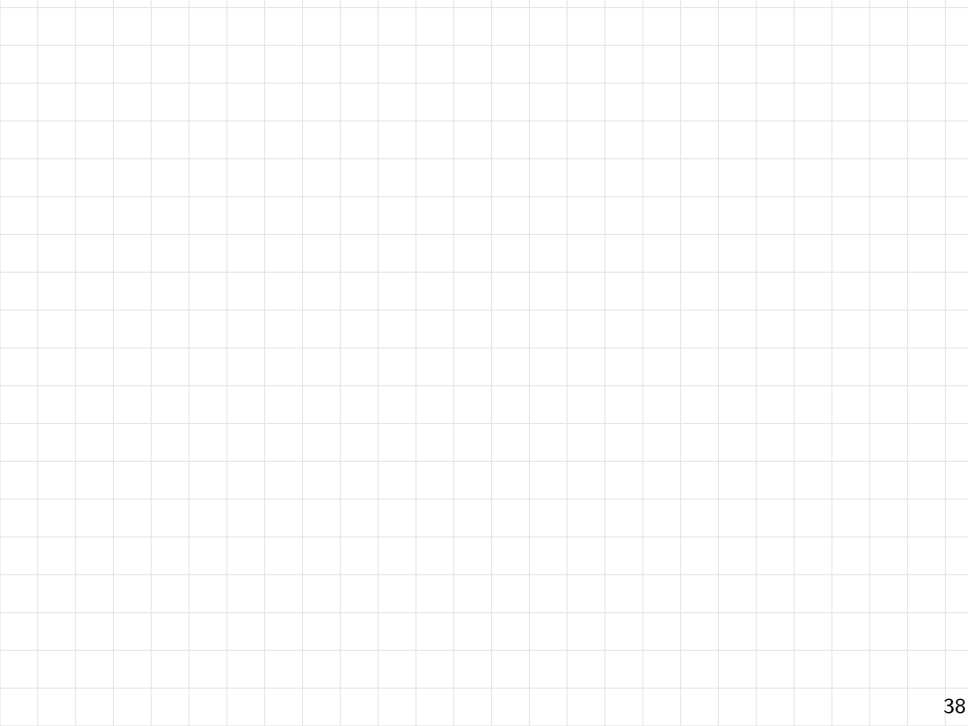
We also define:

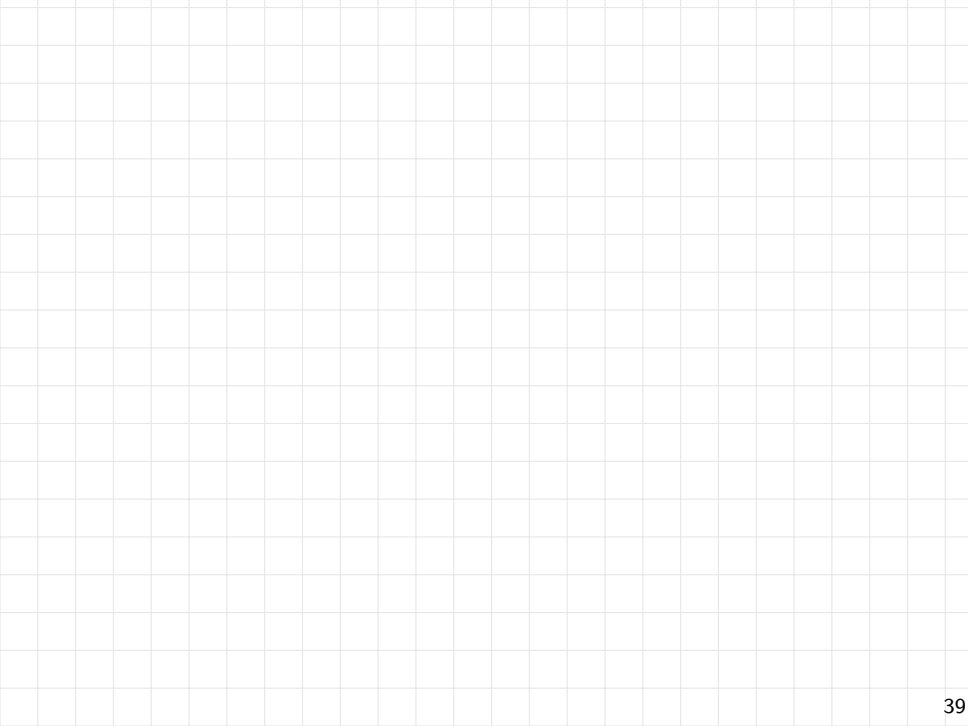
- $\Phi_{\text{kcenter}}^{\text{opt}}(P, k) = \Phi_{\text{kcenter}}(P, S^*)$.
- $\Phi_{\text{kmeans}}^{\text{opt}}(P, k) = \Phi_{\text{kmeans}}(P, S^*)$.
- $\Phi_{\text{kmedian}}^{\text{opt}}(P, k) = \Phi_{\text{kmedian}}(P, S^*)$.

Why do we require only the centers as output?

Can a suitable partition be recovered from the centers?





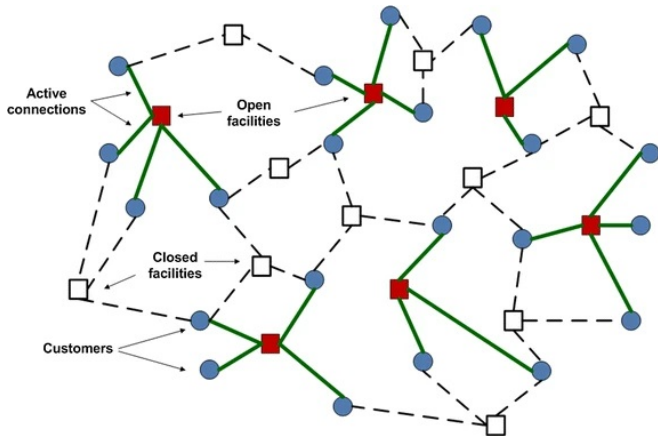


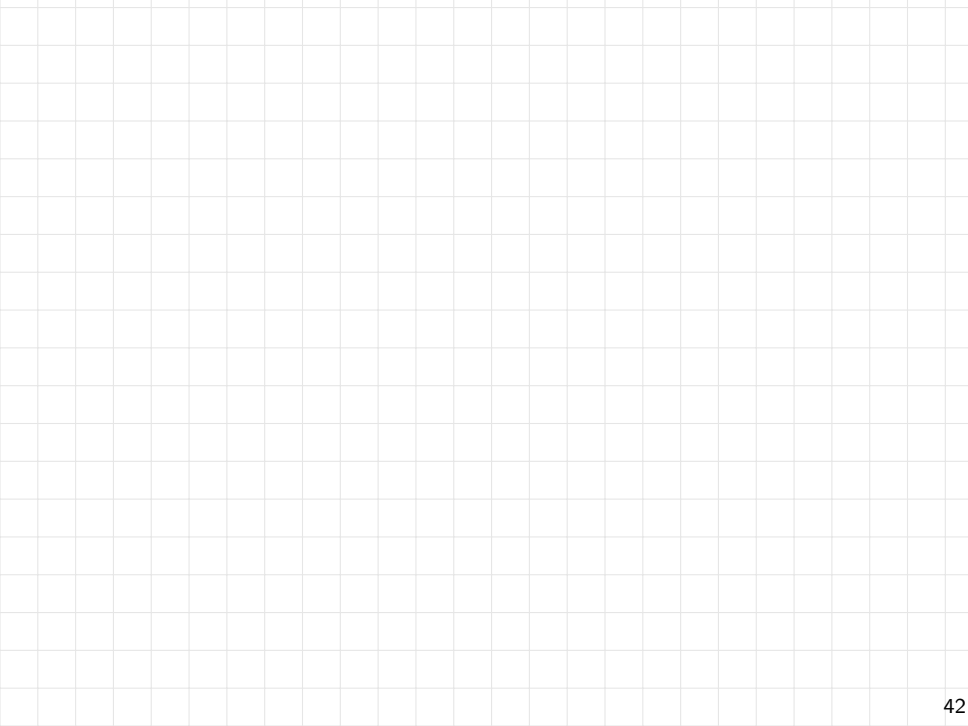
Observations

- The k-center/means/median problems are **NP-hard** $\Rightarrow$ it is impractical to search for the optimal solution S^* .
- There are several efficient approximation algorithms that in practice return good-quality solutions. However, **dealing efficiently with large inputs is still a challenge!**
- **How do we decide which problem suites better our needs?**
 - **k-center** is useful when we need to guarantee that *every point* is close to a center (e.g., if centers represent locations of crucial facilities or if they are needed as "representatives" for the data). As we will see later, it is sensitive to noise.
 - **k-means/median** provide guarantees on the average (square) distances. k-means is more sensitive to noise but there exist faster algorithms to solve it.

Observations

- k-center and k-median belong to the family of **facility-location problems**





k-center

Farthest-First Traversal: algorithm

- Popular 2-approximation sequential algorithm developed by T.F. Gonzalez [Theoretical Computer Science, 38:293-306, 1985]
- Simple and, somewhat fast, implementation when the input fits in main memory.
- Powerful primitive for extracting samples for further analyses

Input Set P of N points from a metric space (M, d) , integer $k > 1$

Output A set S of k centers which is a good solution to the k -center problem on P (i.e., $\Phi_{\text{kcenter}}(P, S)$ “close” to $\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$)

Farthest-First Traversal: algorithm

```
 $S \leftarrow \{c_1\}$  //  $c_1 \in P$  arbitrary point  
for  $i \leftarrow 2$  to  $k$  do  
    Find the point  $c_i \in P - S$  that maximizes  $d(c_i, S)$   
     $S \leftarrow S \cup \{c_i\}$   
return  $S$ 
```

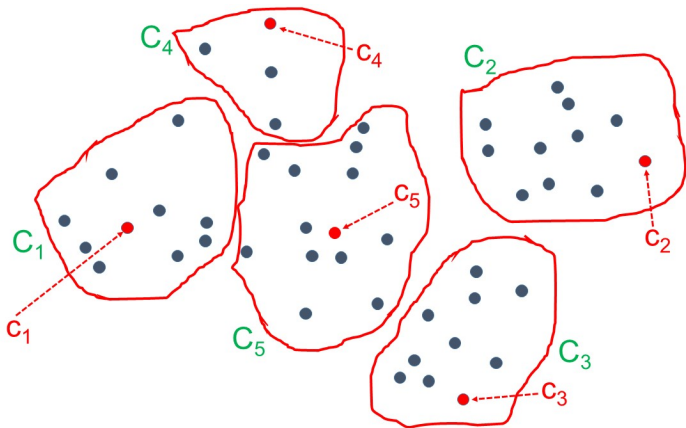
Observation: The best clustering around the centers of S can be computed by invoking $\text{Assign}(P, S)$. In fact, *the assignment of each point to the closest center can be easily maintained in every iteration of the for-loop.*

Farthest-First Traversal: example ($k = 5$)

Pointset P



Farthest-First Traversal: example ($k = 5$)



Exercise

Show that the Farthest-First Traversal algorithm can be implemented to run in $O(N \cdot k)$ time.

Hint: make sure that in each iteration i of the for-loop each point $p \in P - S$ knows its closest center among $c_1, c_2, \dots, c_{i-1}$ and the distance from such a center.

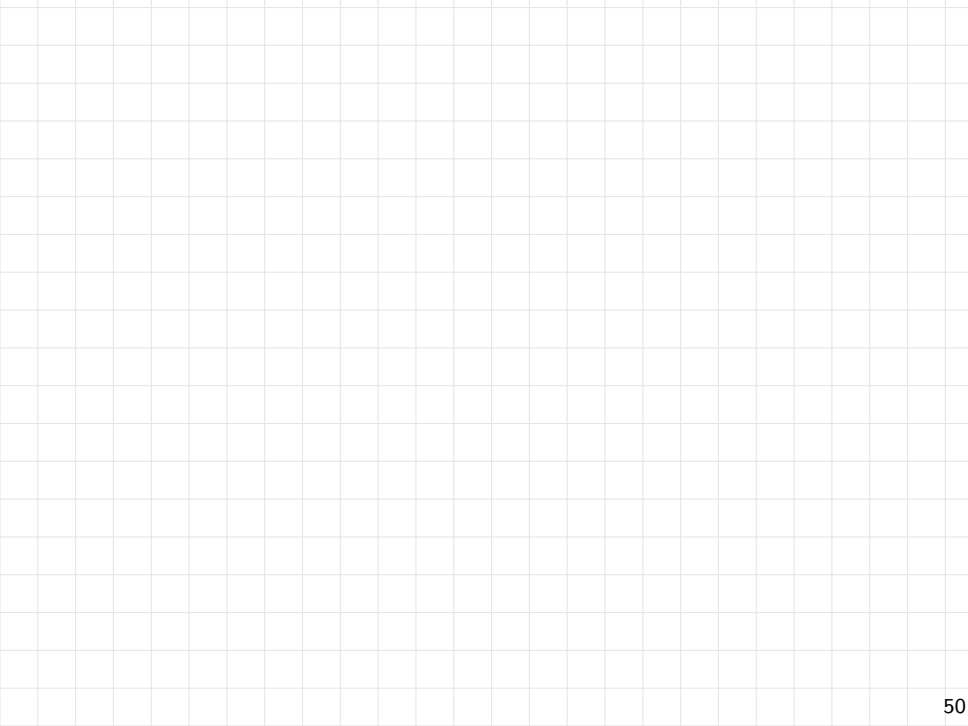
Farthest-First Traversal: analysis

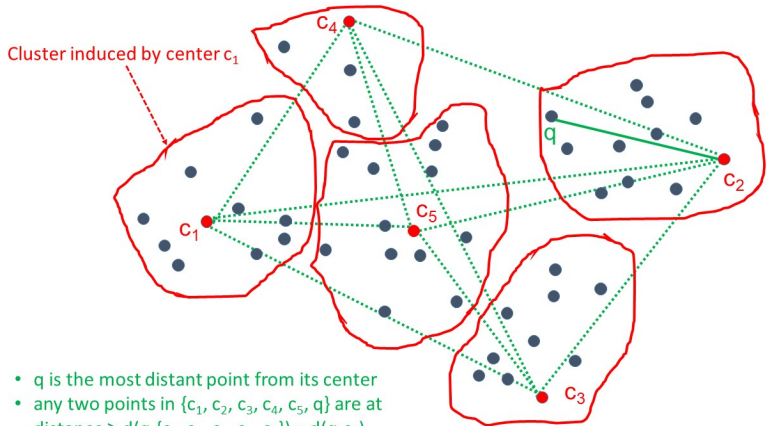
Theorem

Let S be the set of centers returned by running Farthest-First Traversal on P . Then:

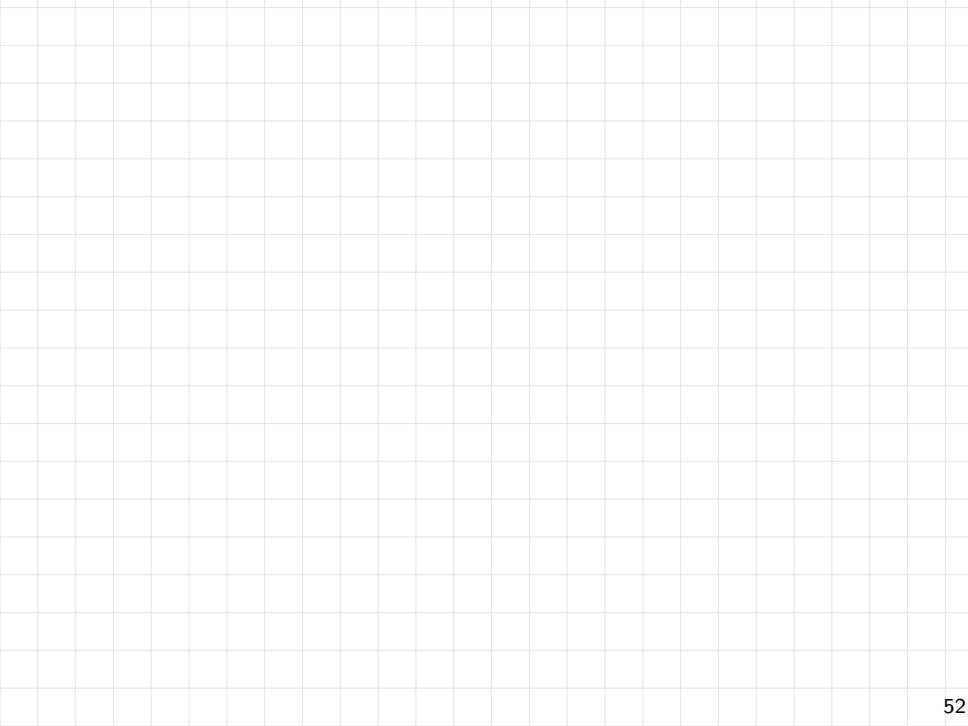
$$\Phi_{\text{kcenter}}(P, S) \leq 2 \cdot \Phi_{\text{kcenter}}^{\text{opt}}(P, k).$$

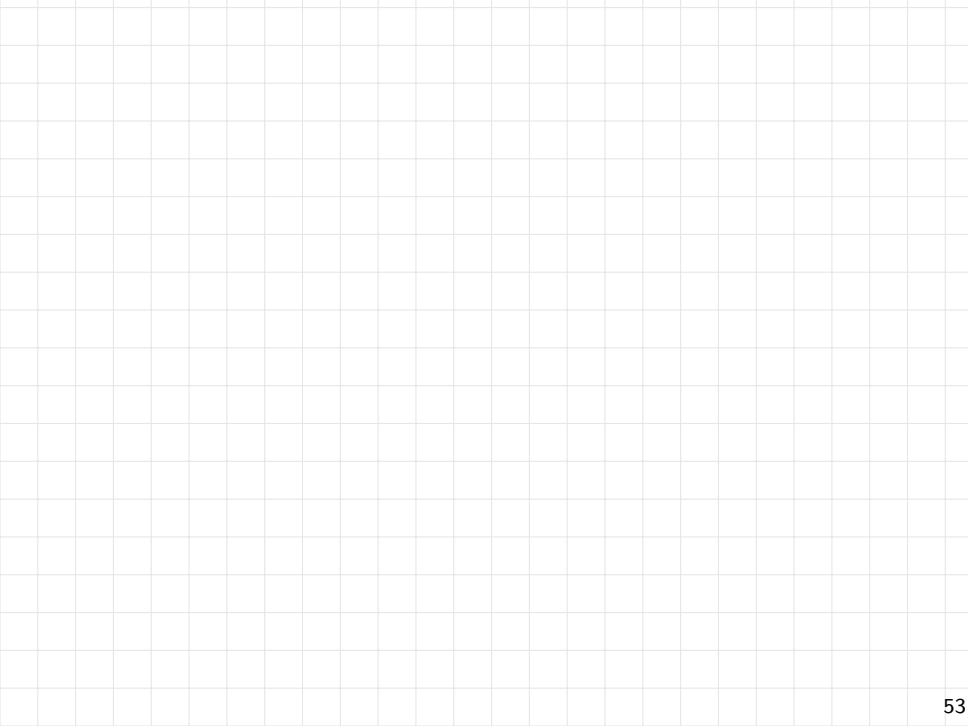
That is, Farthest-First Traversal is a 2-approximation algorithm.

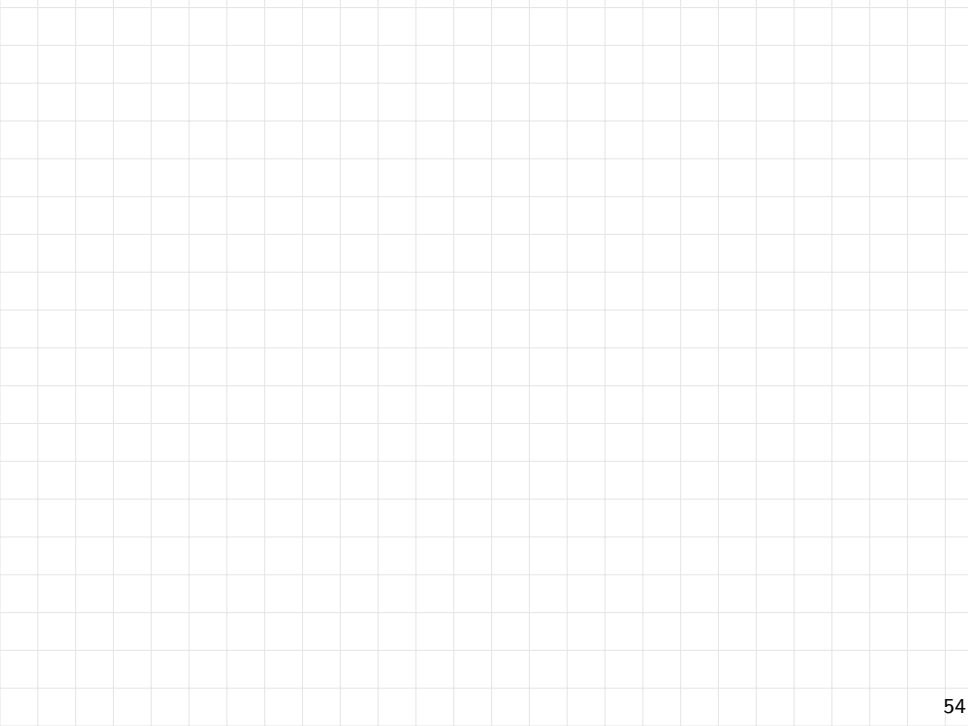


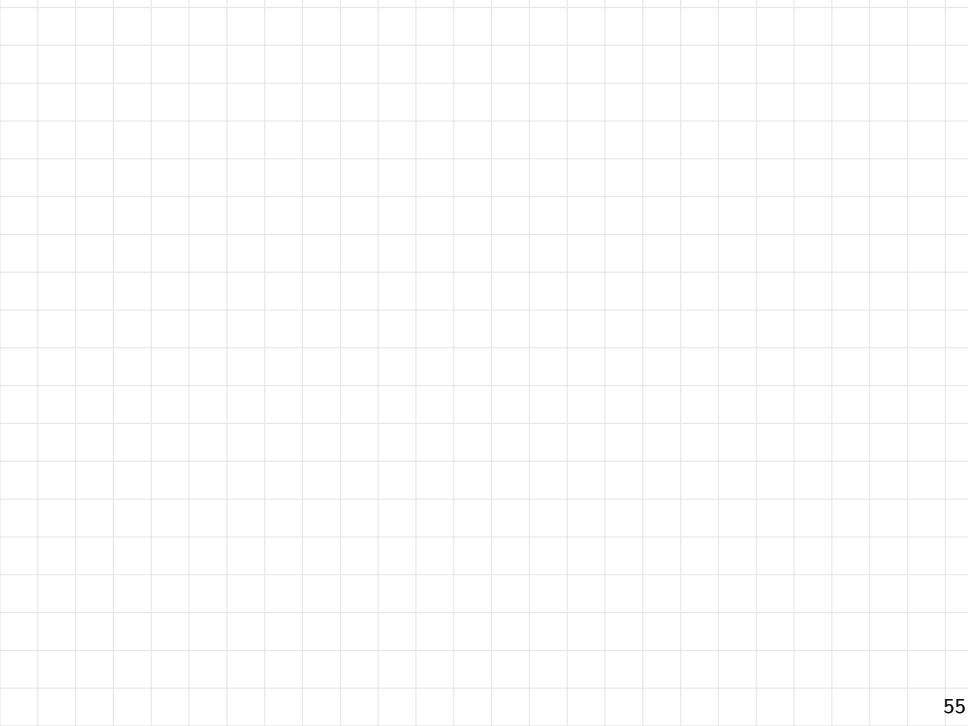


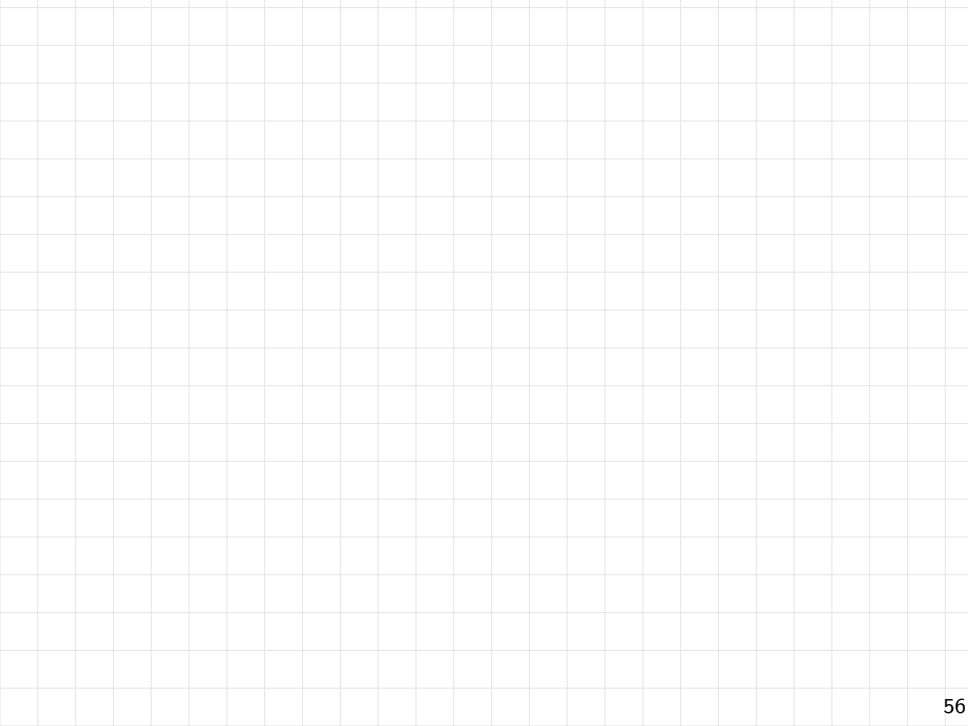
- q is the most distant point from its center
- any two points in $\{c_1, c_2, c_3, c_4, c_5, q\}$ are at distance $\geq d(q, \{c_1, c_2, c_3, c_4, c_5\}) = d(q, c_2)$

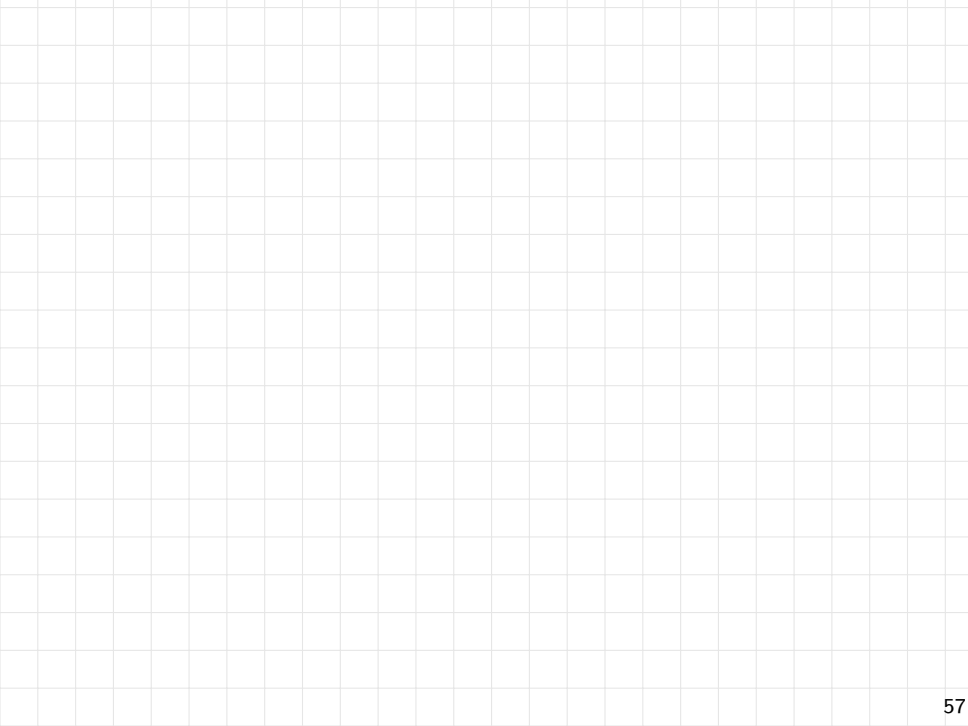








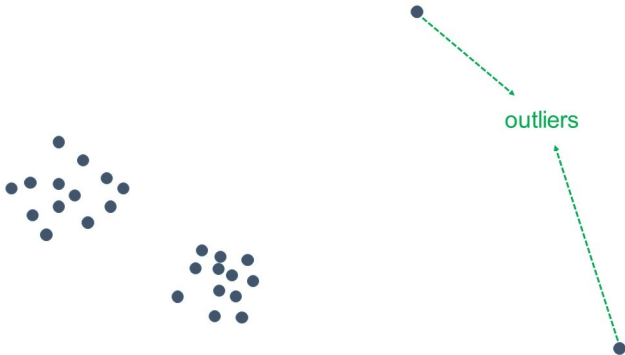




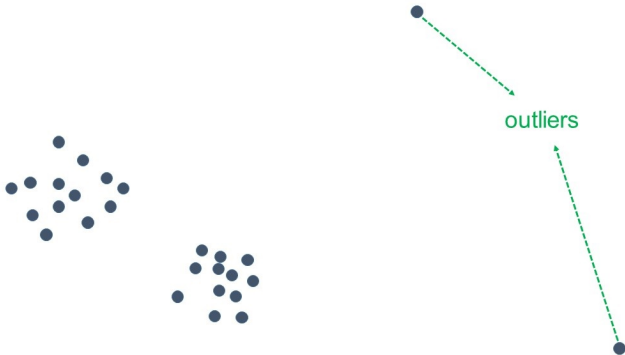
Observations on k-center clustering

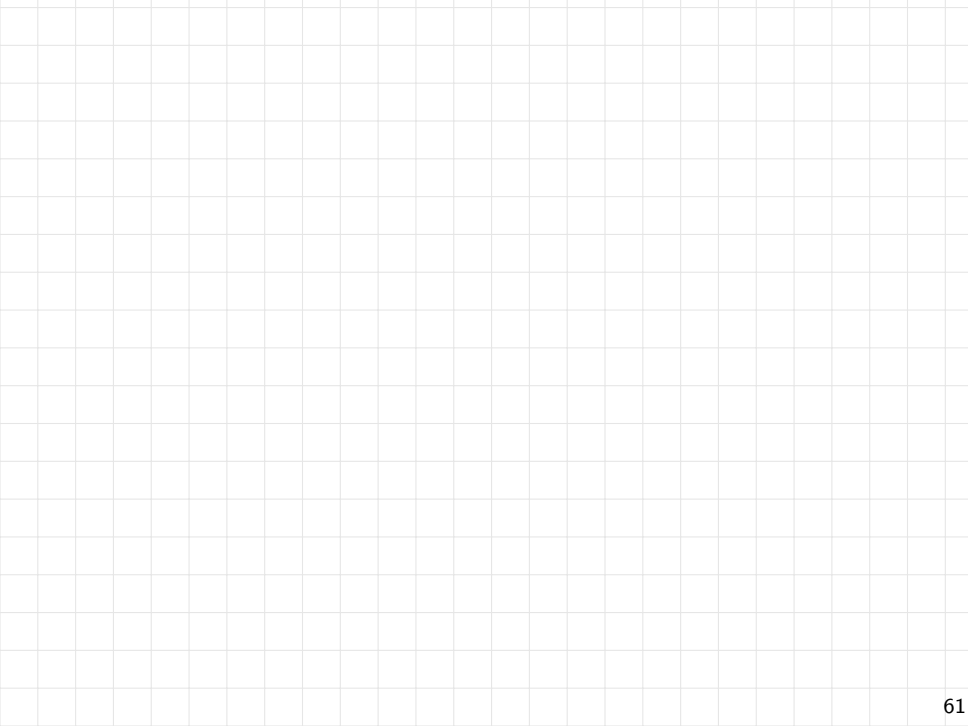
- The k-center objective focuses on worst-case distance of points from their closest centers.
- Farthest-First Traversal's approximation guarantees are almost the best one can obtain in practice. It was proved that computing a c -approximate solution to k-center is NP-hard for any fixed $c < 2$.
- The k-center objective is very sensitive to noise. For noisy datasets (e.g., with outliers), the clustering which optimizes the k -center objective may obfuscate some “natural” clustering inherent in the data. See example in the next slide.

Example: noisy pointset



Example: noisy pointset





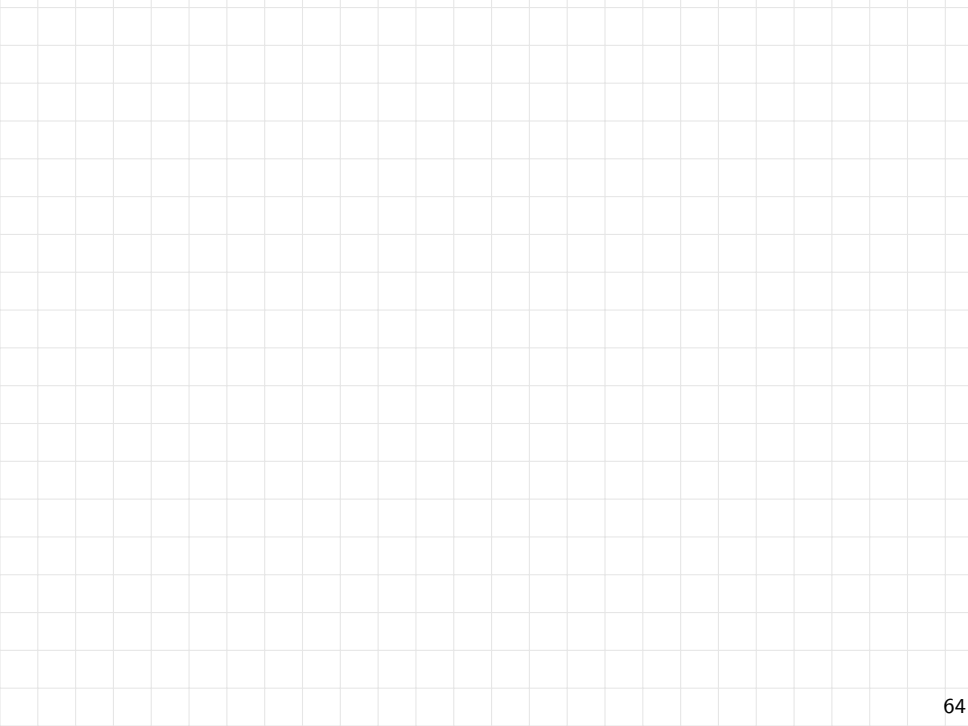
k-center clustering for big data

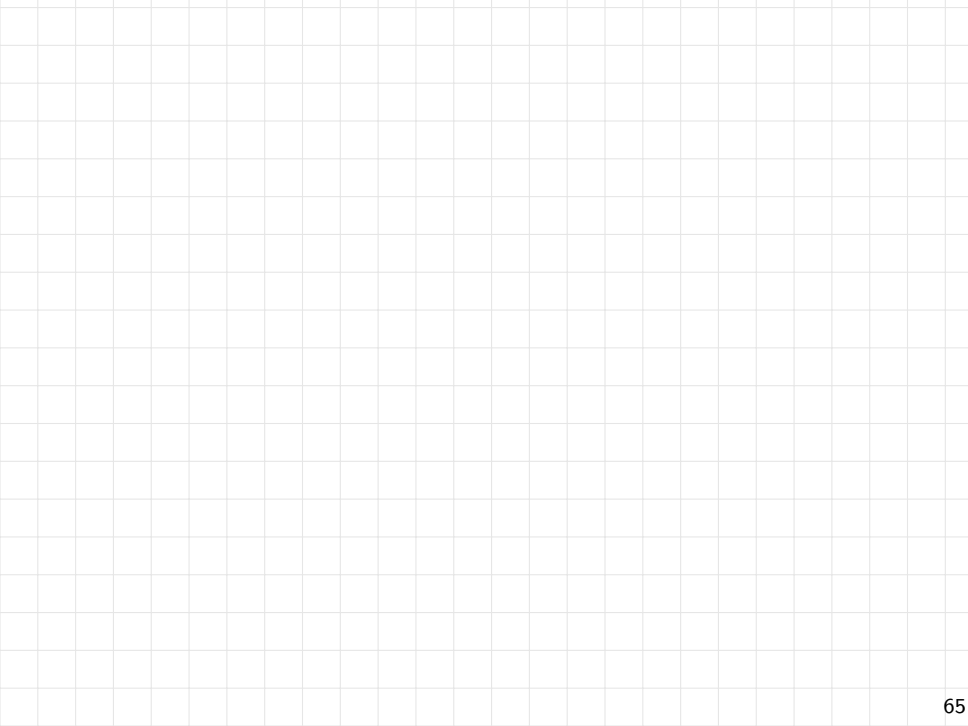
Observation. Farthest-First Traversal requires $k - 1$ scans of the pointset P : impractical for massive P and k not small.

How can we compute a “good” solution to k-center for a pointset P which is too large for a single machine?

We resort to the composable coresset technique.

Composable coresets for k-center clustering





MapReduce-Farthest-First Traversal

Let P be a set of N points (N large!) from a metric space (M, d) , and let $k > 1$ be an integer.

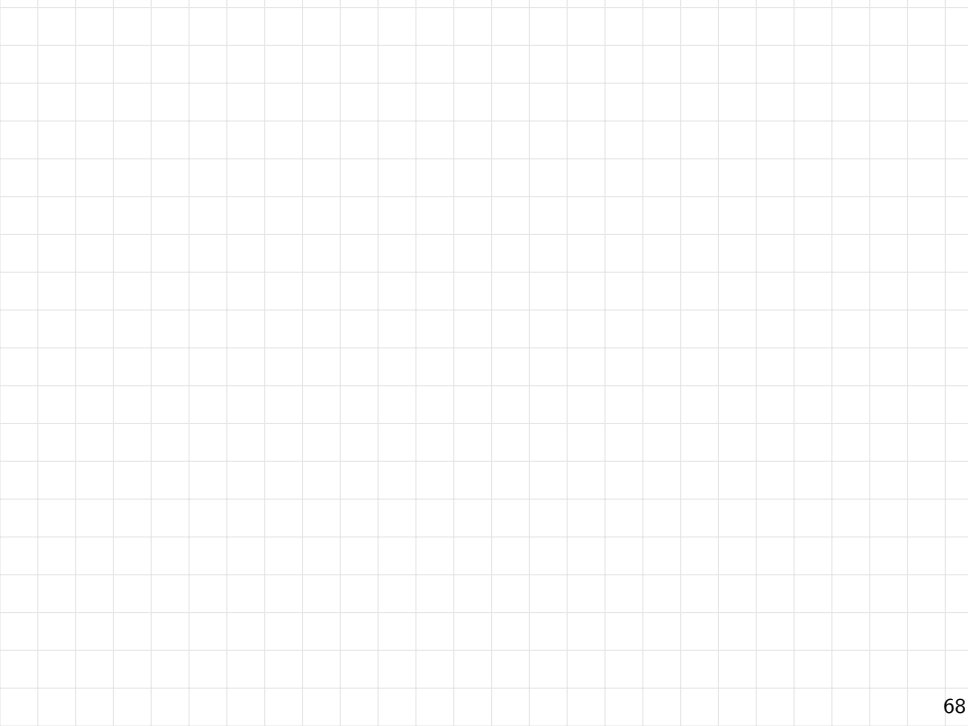
Algorithm MR-Farthest-First Traversal

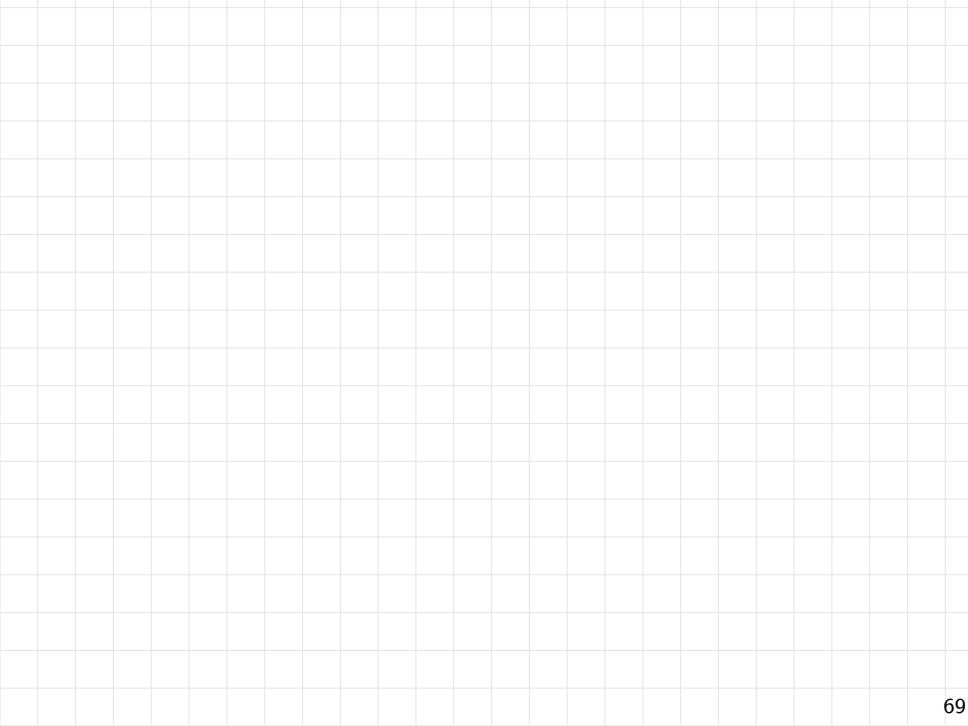
- **Round 1:**
 - Map Phase: Partition P arbitrarily into ℓ subsets of equal size $P_1, P_2, \dots, P_\ell$.
 - Reduce Phase: for every $i \in [1, \ell]$ separately, run Farthest-First Traversal on P_i to determine a set $T_i \subseteq P_i$ of k centers.
- **Round 2:**
 - Map Phase: empty.
 - Reduce Phase: gather the **coreset** $T = \cup_{i=1}^{\ell} T_i$ (of size $\ell \cdot k$) and return a set $S = \{c_1, c_2, \dots, c_k\}$ of k centers, computed by running Farthest-First Traversal on T using a single reducer.

Analysis of MR-Farthest-First Traversal

Theorem

The 2-round MR-Farthest-First Traversal algorithm can be implemented using local space $M_L = O(\sqrt{Nk})$ and aggregate space $M_A = O(N)$.





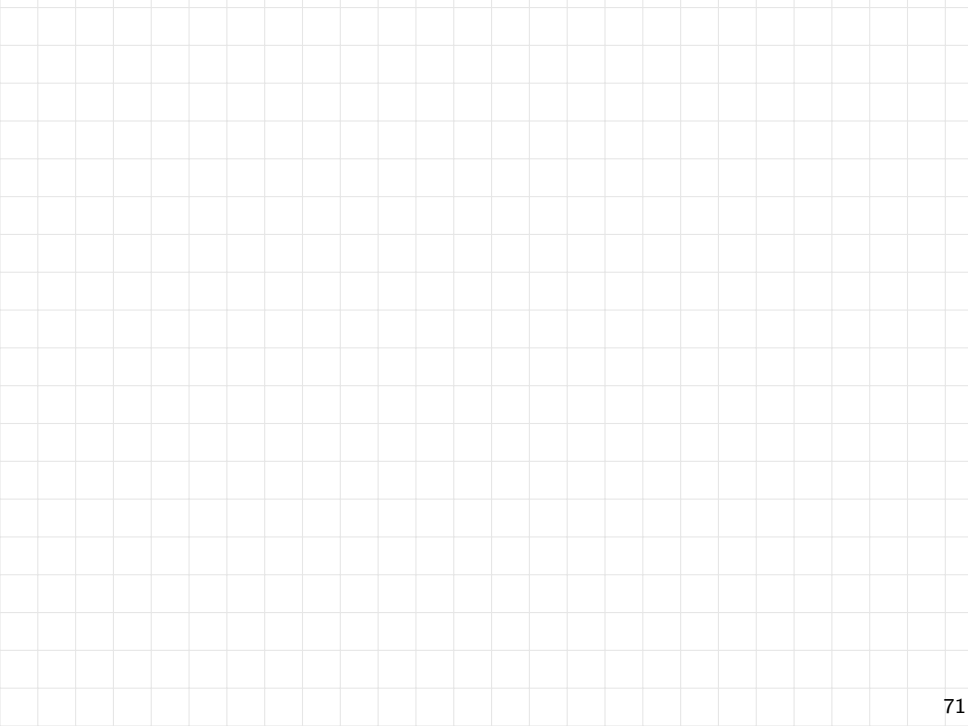
Let T be the union of the coresets T_i computed by MR-Farthest-First Traversal on input P . The following lemma establishes the quality of T .

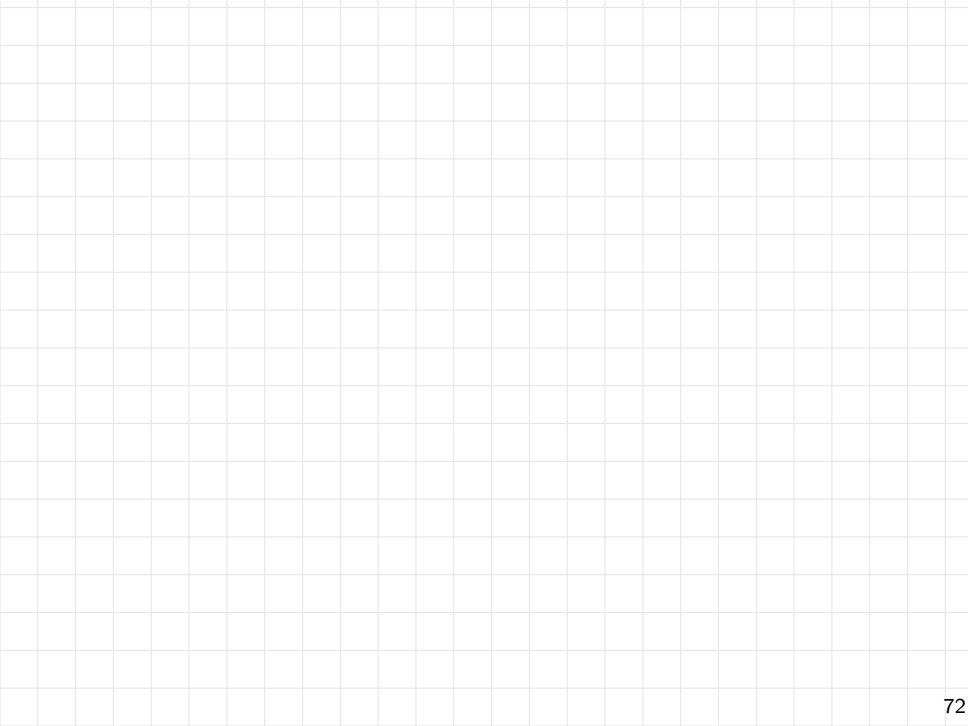
Lemma

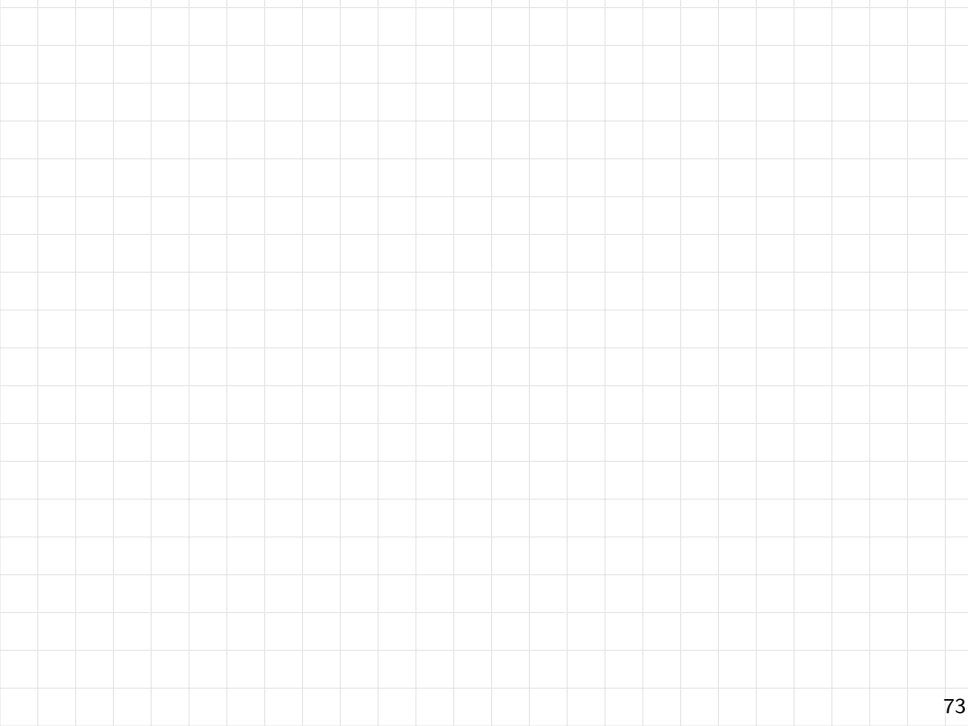
For every $x \in P$ we have

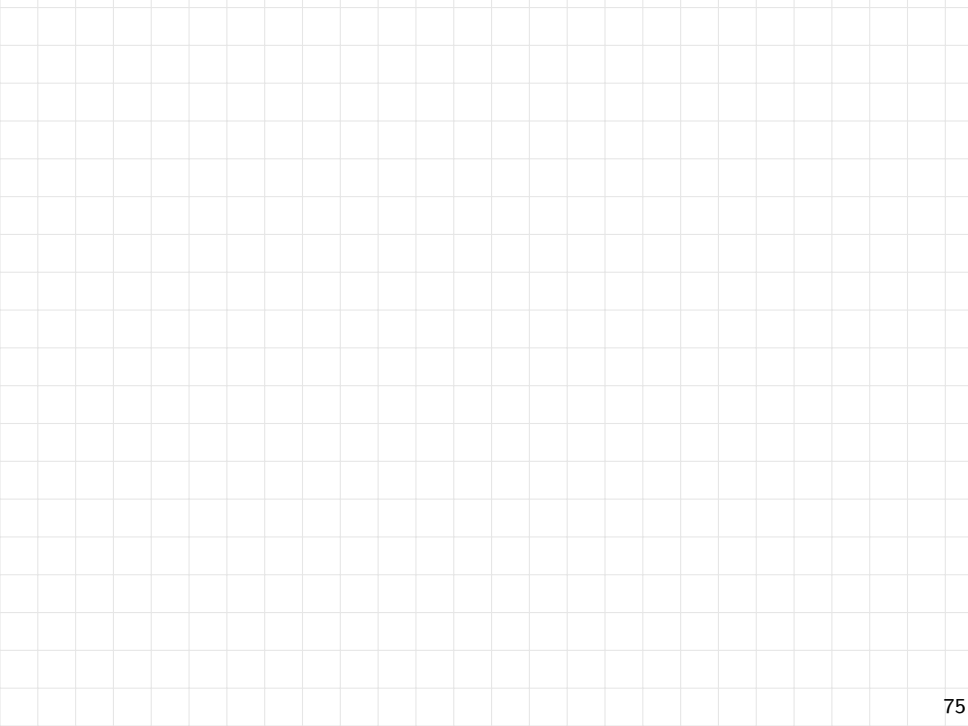
$$d(x, T) \leq 2 \cdot \Phi_{\text{kcenter}}^{\text{opt}}(P, k).$$

Observation. The lemma gives a quantitative characterization of how well the coreset T represents the input P , with respect to the k -center objective.









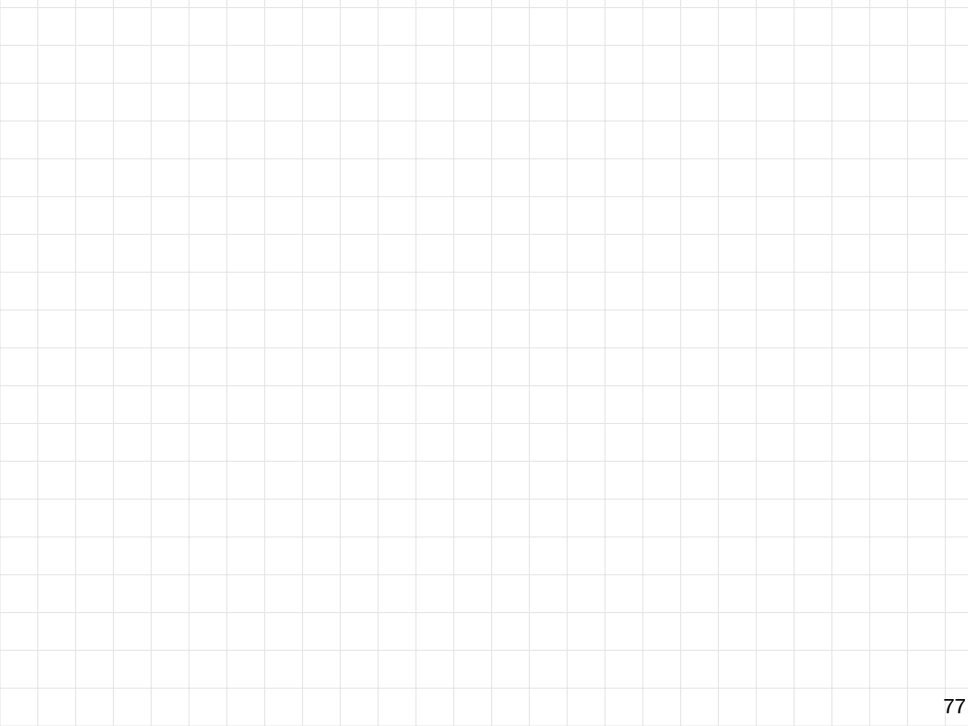
Analysis of MR-Farthest-First Traversal

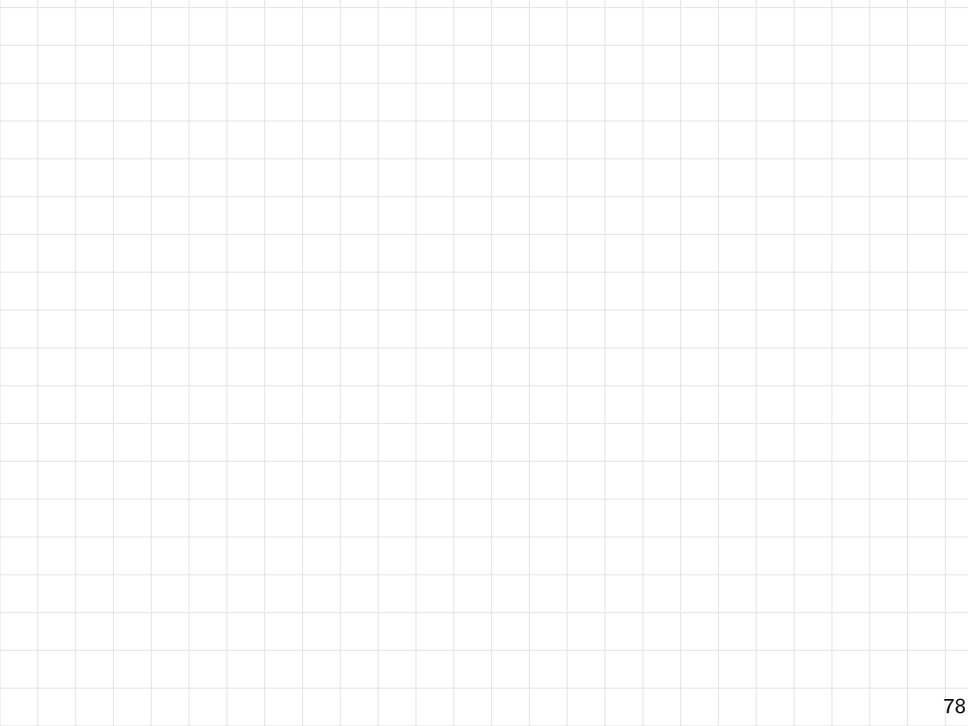
Theorem

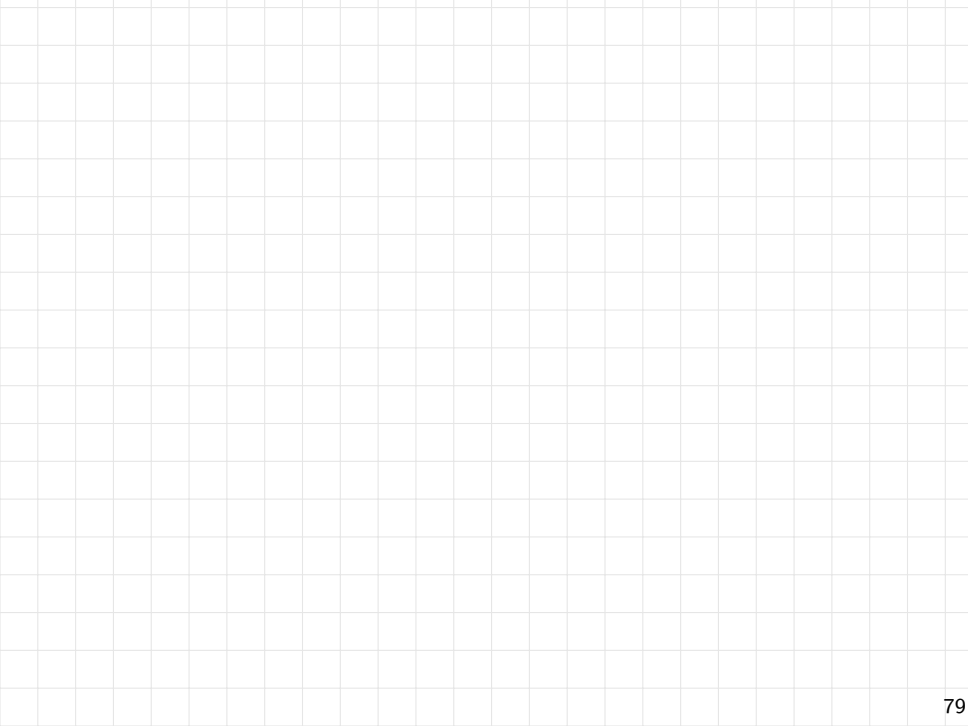
Let S be the set of k centers returned by running MR-Farthest-First Traversal on P . Then:

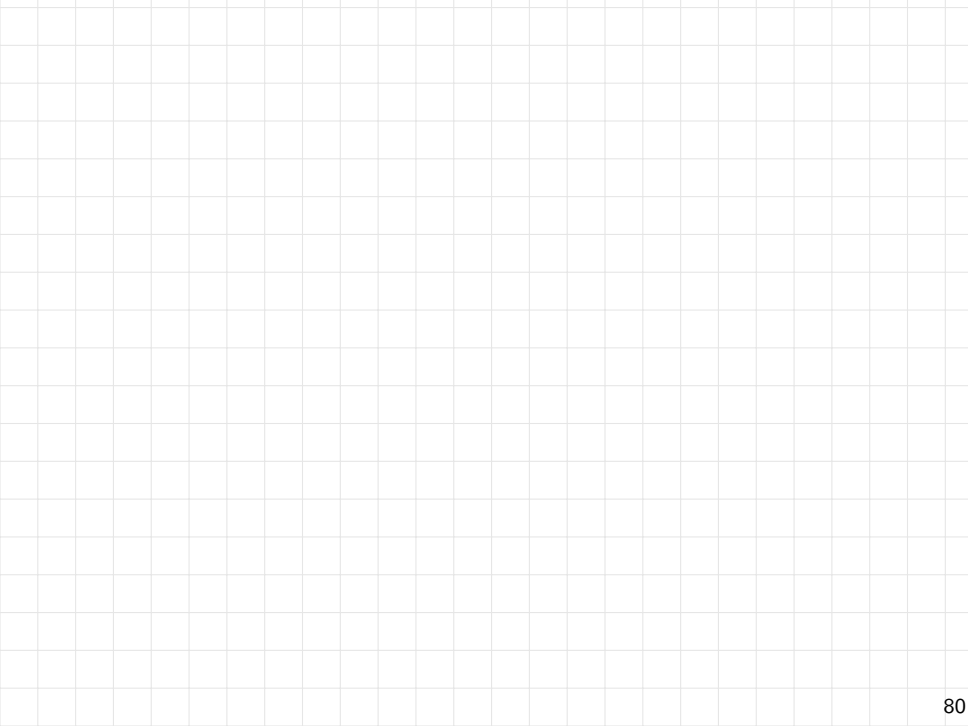
$$\Phi_{\text{kcenter}}(P, S) \leq 4 \cdot \Phi_{\text{kcenter}}^{\text{opt}}(P, k).$$

That is, MR-Farthest-First Traversal is a 4-approximation algorithm.







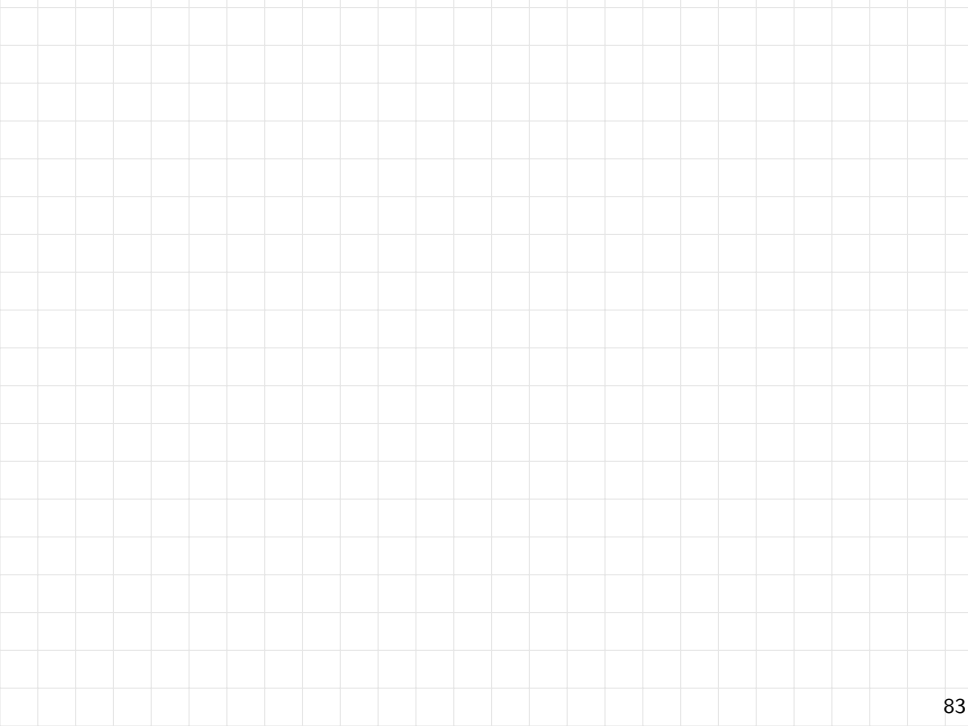


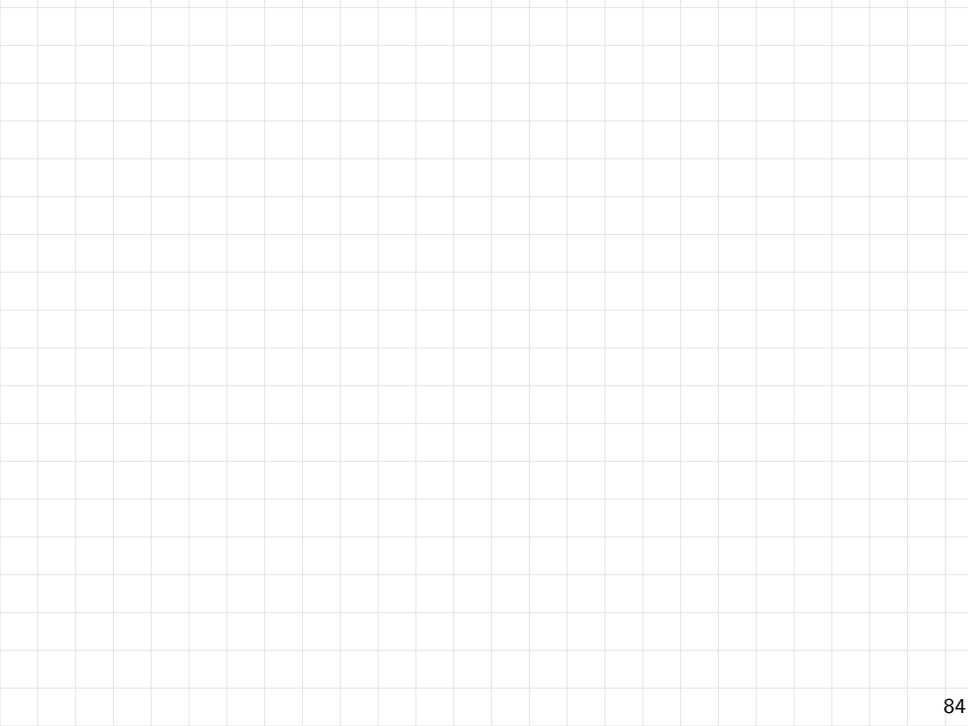
Observations on MR-Farthest-First Traversal

- Farthest-First Traversal provides good coresets T_i 's, hence a good final coreset T since it ensures that any point not belonging to T is well represented by some coreset point.
- MR-Farthest-First Traversal is able to handle very large pointsets and the the final approximation is not too far from the best achievable one.
- Observe that for any optimization problem, no MR algorithm can attain better accuracy than the best sequential algorithm, since all MR algorithms become sequential algorithms when executed on one processing element.

Low-dimensional pointsets

When P has low dimensionality the quality of the solution returned by MR-Farthest-First Traversal can be made arbitrarily close to 2 by selecting a slightly larger coreset, while still ensuring sublinear local space and linear aggregate space.

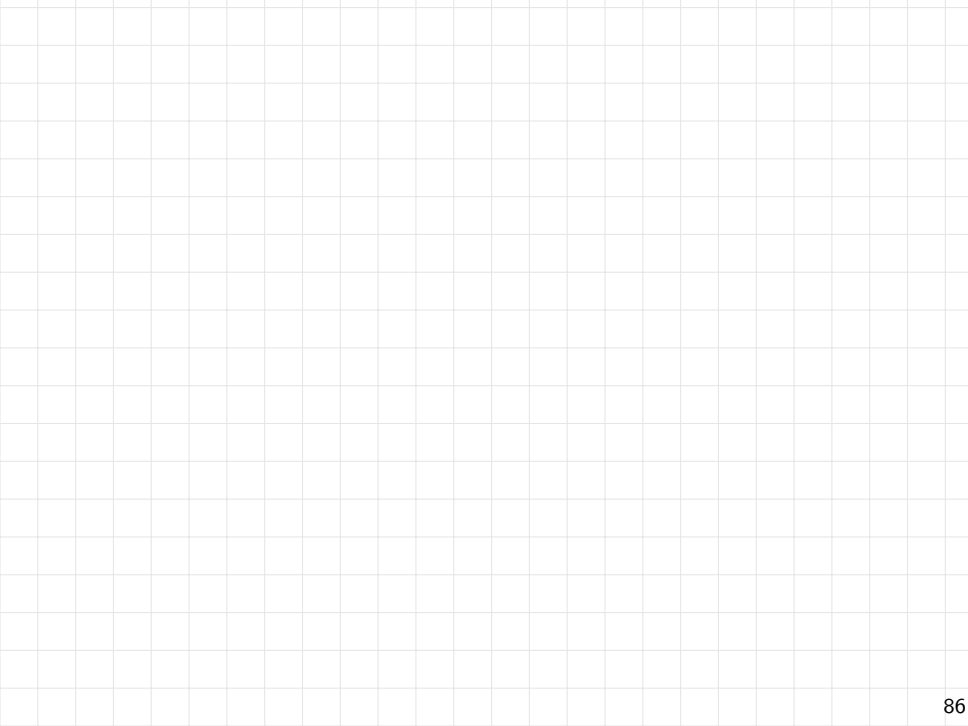


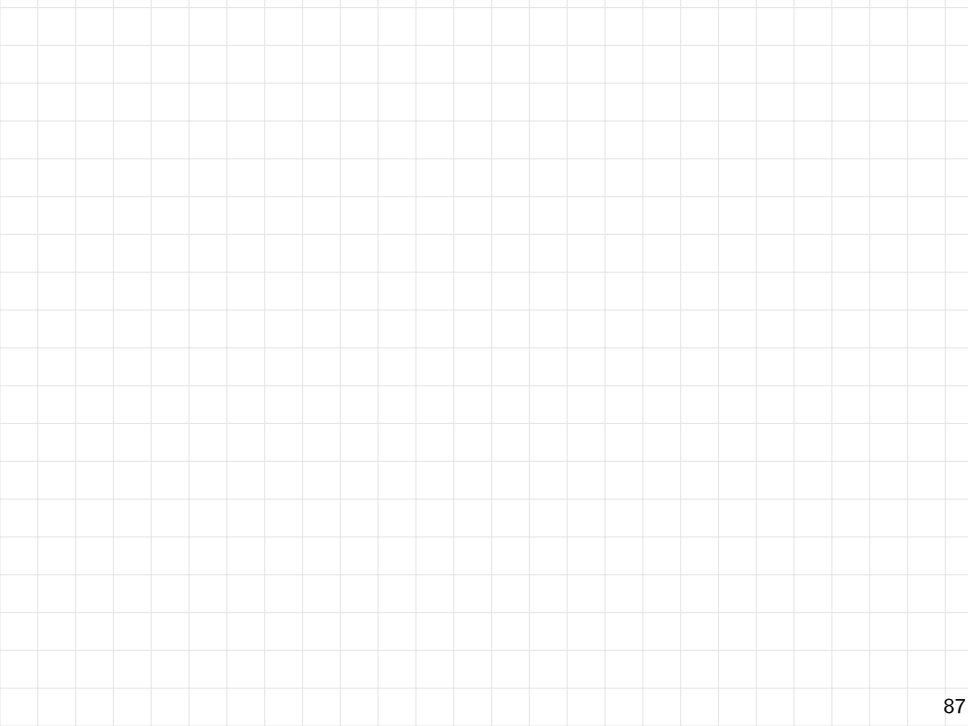


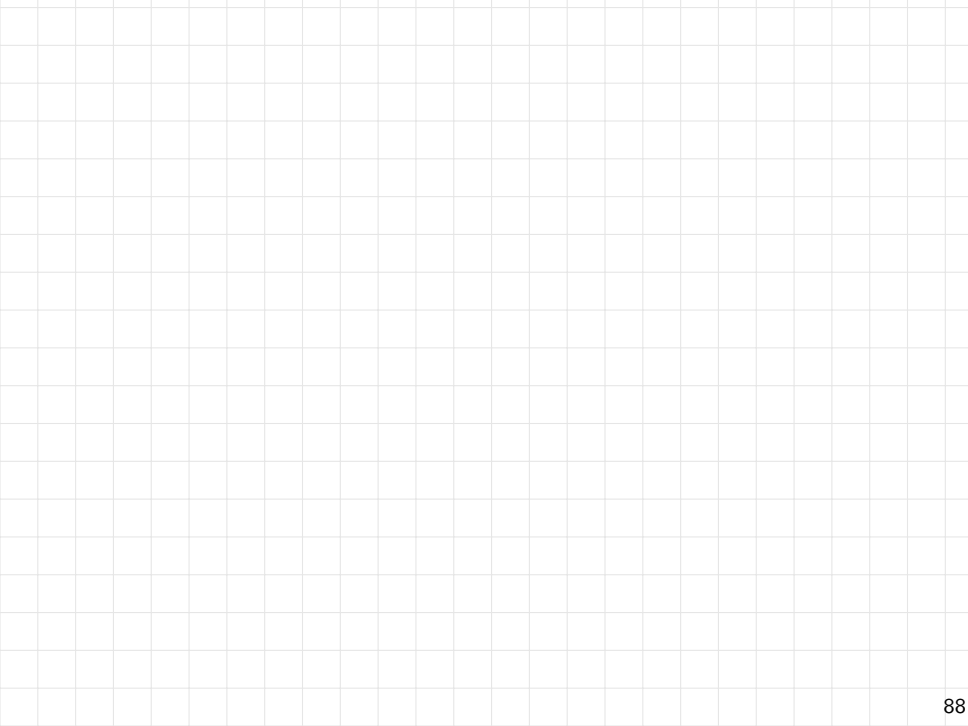
Does a random sample provide a good coreset?

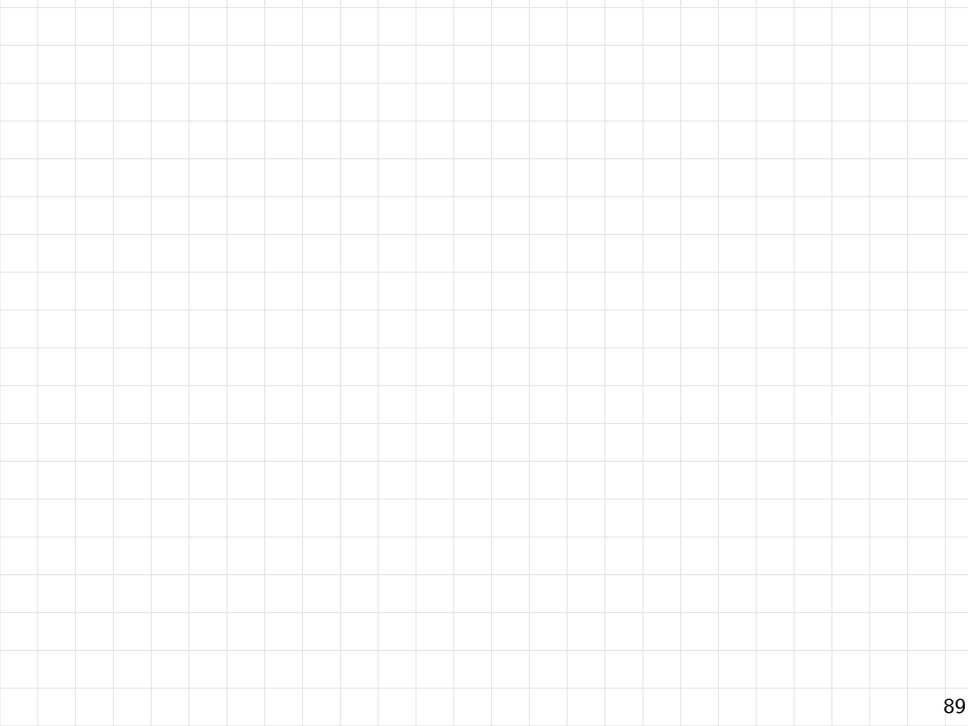
Let $T \subseteq P$ be a coreset of $|T| = \sqrt{Nk}$ points, selected at random from P independently, with replacement and with uniform probability. Consider a set S of k centers computed by running Farthest-First Traversal on T .

Is S a good solution to k -center on P ?









Exercise

Let P be a set of N points in a metric space (M, d) , and let $T \subseteq P$ be a coreset of $|T| > k$ points such that for each $x \in P$ we have $d(x, T) \leq \epsilon \Phi_{\text{kcenter}}^{\text{opt}}(P, k)$, for some $\epsilon \in (0, 1)$. Let S be the set of k centers obtained by running the Farthest-First Traversal algorithm on T . Prove an upper bound to $\Phi_{\text{kcenter}}(P, S)$ as a function of ϵ and $\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$.

Exercise

Let P be a set of points in a metric space (M, d) , and let $T \subseteq P$. For any $k < |T|, |P|$, show that $\Phi_{\text{kcenter}}^{\text{opt}}(T, k) \leq 2\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$. Is the bound tight?

Exercise

Consider a set P of N points from a metric space, represented by the key-value pairs (i, x_i) , with $0 \leq i < N$ and $x_i \in P$. Let S be a set of k centers from the same metric space but *not belonging to* P , which are stored in a global array $S[1], S[2], \dots, S[k]$. For $\ell = 1, 2, \dots, k$, denote with $P^{(\ell)}$ the set of points of P whose closest center is $S[\ell]$. Assume that each point in P has exactly one closest center in S , and that $k = O(\sqrt{N})$.

- 1 Design a 2-round MapReduce algorithm that, given P and S , computes a coreset $T \subset P$ containing one point from each non-empty $P^{(\ell)}$.
- 2 Analyze the local and aggregate space required by your algorithm. For full score, your algorithm must require $o(N)$ local space and linear aggregate space.
- 3 Show how to use T to approximate the value $\Delta(P) = \max_{x,y \in P} d(x,y)$, and estimate the quality of the approximation in terms of $R = \max_{x \in P} d(x, S)$. (Recall that each center $S[\ell]$ is not in P , hence it cannot be in T .)

Exercise

Let P be a set of N points in a metric space (M, d) , and let $\mathcal{C} = (C_1, C_2, \dots, C_k; S)$ be a k -clustering of P , with $S = \{c_1, c_2, \dots, c_k\}$. Initially, each point $q \in P$ is represented by a pair $(\text{ID}(q), (q, c(q)))$, where $\text{ID}(q)$ is a distinct key in $[0, N - 1]$ and $c(q) \in S$ is the center of the cluster of q .

- 1 Design a 2-round MapReduce algorithm that for each cluster center c_i determines the most distant point among those belonging to the cluster C_i (ties can be broken arbitrarily).
- 2 Analyze the local and aggregate space required by your algorithm. Your algorithm must require $o(N)$ local space and $O(N)$ aggregate space.

Exercise

Let P be a set of N *bicolored points* from a metric space, partitioned into k clusters $C_1, C_2, \dots, C_k$. Each point $x \in P$ is initially represented by the key-value pair $(ID_x, (x, i_x, \gamma_x))$, where ID_x is a distinct key in $[0, N - 1]$, i_x is the index of the cluster which x belongs to, and $\gamma_x \in \{0, 1\}$ is the color of x .

- 1 Design a 2-round MapReduce algorithm that for each cluster C_i checks whether all points of C_i have the same color. The output of the algorithm must be the k pairs (i, b_i) , with $1 \leq i \leq k$, where $b_i = -1$ if C_i contains points of different colors, otherwise b_i is the color common to all points of C_i .
- 2 Analyze the local and aggregate space required by your algorithm. Your algorithm must require $o(N)$ local space and $O(N)$ aggregate space.